

SIMATS ENGINEERING
ASSESSMENT TOOL 1 - Medium
Coding Challenge

COURSE CODE: ITA04 COURSE NAME: Statistics with R Programming

COURSE OUTCOME COVERED

CO2: Analyze datasets using Exploratory Data Analysis techniques including measures of central tendency, dispersion, bivariate analysis, and statistical tests such as T-test, ANOVA, and chi-square. (BL4-BL5)

Assessment Tool Weightage: 25%

QUESTIONS: 5

Total Marks: 50

Coding Challenge

Code of Conduct:

I Ck Afroz(192311108) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Ck Afroz

Coding Challenge

1. Write an R program to declare variables of each basic data type (numeric, integer, character, logical, complex) and print their class using `class()`.
2. Create a list containing a vector, a matrix, and a character string. Write a program to access and print each element of the list individually.
3. Write a program to create a 4x4 matrix of integers and perform matrix addition, subtraction, and multiplication with another 4x4 matrix.
4. Create a 3-dimensional array of size 2x3x4 filled with random numbers, then write a program to extract and print a specific slice of the array.
5. Define an S3 class called Student with fields name, age, and marks, and write a method to display student details. Then implement the same using an S4 class and compare the outputs.
6. Write an R program using a for loop to print the Fibonacci series up to n terms, then rewrite the same using a while loop.
7. Write a program using nested if-else statements to categorize a student's grade based on marks entered by the user.
8. Write a recursive R function to calculate the factorial of a number, and another recursive function to find the GCD of two numbers.
9. Write a function in R with default argument values that calculates the simple interest, and demonstrate calling it with and without passing all arguments.
10. Write a program to loop over a non-vector set (like a list of mixed data types) and print each element along with its data type.
11. Create a data frame with columns StudentID, Name, Marks, and Grade for 10 students. Write a program to add a new column Result based on a condition on Marks.
12. Write a program to merge two data frames — one containing student names and IDs, and another containing student IDs and marks — using a common key column.
13. Write a program to demonstrate melting and casting of a wide-format data frame into long format and back, using the reshape2 package.
14. Write an R program to read data from a CSV file into a data frame, clean missing values, and write the cleaned data frame back to a new CSV file.
19. Write a program to create an atomic vector of 10 numeric values and demonstrate at least 5 different vector operations (sum, mean, sorting, reversing, indexing).
20. Write a program to create two logical vectors and demonstrate AND, OR, NOT, and XOR operations on them element-wise.
21. Write a program to create a factor variable from a character vector representing blood groups, and display its levels and frequency table.

22. Write a program to create a named list representing an employee record (name, id, salary, department) and update one field's value.
23. Write a program to demonstrate the difference between a Reference Class and an S4 class by creating a BankAccount object with deposit() and withdraw() methods.
24. Write a program to create a character vector of names, then convert it to uppercase, lowercase, and count the number of characters in each name.
25. Write a program using a for loop to check whether each number in a given vector is prime or not.
26. Write a program to reverse a string using recursion (without using the built-in reverse function).
27. Write a program using while loop and break/next statements to print only even numbers between 1 and 50, skipping multiples of 10.
28. Write a recursive function to compute the sum of digits of a given number.
29. Write a program with a function that returns multiple values (as a list) — minimum, maximum, and average of a numeric vector — and demonstrate calling it.
30. Write a program to implement a simple calculator using a function with arguments for two numbers and an operator (+, -, *, /), using if-else control statements.
31. Write a recursive R function to compute the nth term of a geometric progression given the first term and common ratio.
32. Write a program to check if a given matrix is symmetric using nested loops and control statements.
33. Write a program to create a data frame of employee records and use apply()/sapply() functions to compute the total salary column-wise.
34. Write a program to sort a data frame of student records by marks in descending order and display the top 5 students.
35. Write a program to filter rows in a data frame based on multiple conditions (e.g., Marks > 60 AND Attendance > 75).
36. Write a program to read a dataset from a text file, split one column into two new columns (e.g., splitting a "Full Name" into First and Last Name), and save the result.
37. Write a program to remove duplicate rows from a data frame and count how many duplicates were removed.
38. Write a program to add, rename, and delete columns in an existing data frame of product sales data.
39. Write a program using a for loop combined with if-else to print Pascal's Triangle up to n rows.
40. Write a program to implement a recursive function that calculates the power of a number (base^{exponent}) without using the ^ operator.
41. Write a program using a while loop to implement a simple guessing game where the user guesses a number until they get it correct, tracking the number of attempts.

42. Write a function in R that explicitly uses `return()` to exit early from a loop when a specific condition is met (e.g., stop searching a vector once a target value is found).
43. Write a recursive function to generate all Fibonacci numbers less than a given limit `n`, and explain why recursion is less efficient than looping for this case.
44. Write a program demonstrating a function that returns a complex object (a list containing a vector, a string, and a boolean) and access its individual components after the call.
45. Write a program to create a data frame from three separate vectors (Name, Department, Salary) and convert it into a matrix, explaining any changes in data type.
46. Write a program to group a data frame by a categorical column (e.g., Department) and compute the average salary for each group using `aggregate()` or `tapply()`.
47. Write a program to write a data frame to a .txt file with a custom delimiter (e.g., "|"), then read it back into R and verify the data matches.
48. Write a program to reshape a data frame from long format to wide format using `dcast()`, based on a dataset containing StudentID, Subject, and Marks.
49. Write a program to identify and count missing values (NA) column-wise in a data frame, then impute them using the column mean.
50. Write a program to join two data frames using `merge()` with an outer join (full join) and explain how missing matches are represented in the result.

R Programming Lab Programs

1. Basic Data Types

```
# Numeric
num <- 10.5

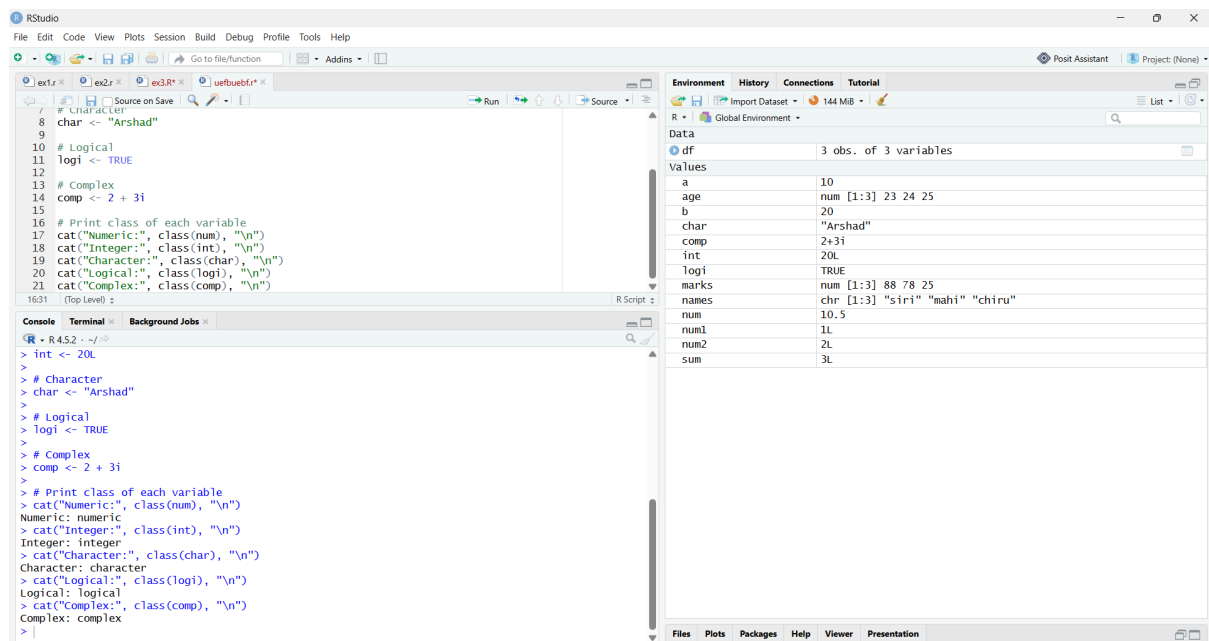
# Integer
int <- 20L

# Character
char <- "Arshad"

# Logical
logi <- TRUE

# Complex
comp <- 2 + 3i

cat("Numeric:", class(num), "\n")
cat("Integer:", class(int), "\n")
cat("Character:", class(char), "\n")
cat("Logical:", class(logi), "\n")
cat("Complex:", class(comp), "\n")
```



2. List Example

```
v <- c(10,20,30,40)
m <- matrix(1:9,nrow=3)
s <- "Welcome to R Programming"
my_list <- list(v,m,s)

cat("Vector:\n"); print(my_list[[1]])
```

```
cat("\nMatrix:\n"); print(my_list[[2]])
cat("\nCharacter String:\n"); print(my_list[[3]])
```

```
# Character String
s <- "Welcome to R Programming"

# Create list
my_list <- list(v, m, s)

# Print each element
cat("Vector:\n")
print(my_list[[1]])

cat("\nMatrix:\n")
print(my_list[[2]])

cat("\nCharacter String:\n")
print(my_list[[3]])
```

```
R - R4.5.2 ~ /
> my_list <- list(v, m, s)
> 
> # Print each element
> cat("Vector:\n")
Vector:
> print(my_list[[1]])
[1] 10 20 30 40
> 
> cat("\nMatrix:\n")
Matrix:
> print(my_list[[2]])
     [,1] [,2] [,3]
[1,]    1     4     7
[2,]    2     5     8
[3,]    3     6     9
> 
> cat("\nCharacter String:\n")
Character String:
> print(my_list[[3]])
[1] "Welcome to R Programming"
> |
```

Environment

Object	Class	Attributes
df	data.frame	3 obs. of 3 variables
m	integer	[1:3, 1:3] 1 2 3 4 5 6 7 8 9
my_list	list	List of 3
a	numeric	10
age	numeric	[1:3] 23 24 25
b	numeric	20
char	character	"Arshad"
comp	numeric	2+3i
int	integer	20L
logi	logical	TRUE
marks	numeric	[1:3] 88 78 25
names	character	[1:3] "siri" "mahi" "chiru"
num	numeric	10.5
num1	integer	1L
num2	integer	2L
s	character	"Welcome to R Programming"
sum	integer	3L
v	numeric	[1:4] 10 20 30 40

3. Matrix Operations

```
A <- matrix(1:16,nrow=4)
B <- matrix(17:32,nrow=4)
```

```
cat("Addition:\n"); print(A+B)
cat("\nSubtraction:\n"); print(A-B)
cat("\nMatrix Multiplication:\n"); print(A %*% B)
```

```
# Matrix B
B <- matrix(17:32, nrow = 4)

# Addition
cat("Addition:\n")
print(A + B)

# Subtraction
cat("\nSubtraction:\n")
print(A - B)

# Matrix Multiplication
cat("\nMatrix Multiplication:\n")
print(A %*% B)
```

```
R - R4.5.2 ~ /
> # Subtraction
> cat("\nSubtraction:\n")
Subtraction:
> print(A - B)
     [,1] [,2] [,3] [,4]
[1,] -16 -16 -16 -16
[2,] -16 -16 -16 -16
[3,] -16 -16 -16 -16
[4,] -16 -16 -16 -16
> 
> # Matrix Multiplication
> cat("\nMatrix Multiplication:\n")
Matrix Multiplication:
> print(A %*% B)
     [,1] [,2] [,3] [,4]
[1,] 538 650 762 874
[2,] 612 740 868 996
[3,] 686 830 974 1118
[4,] 760 920 1080 1240
> |
```

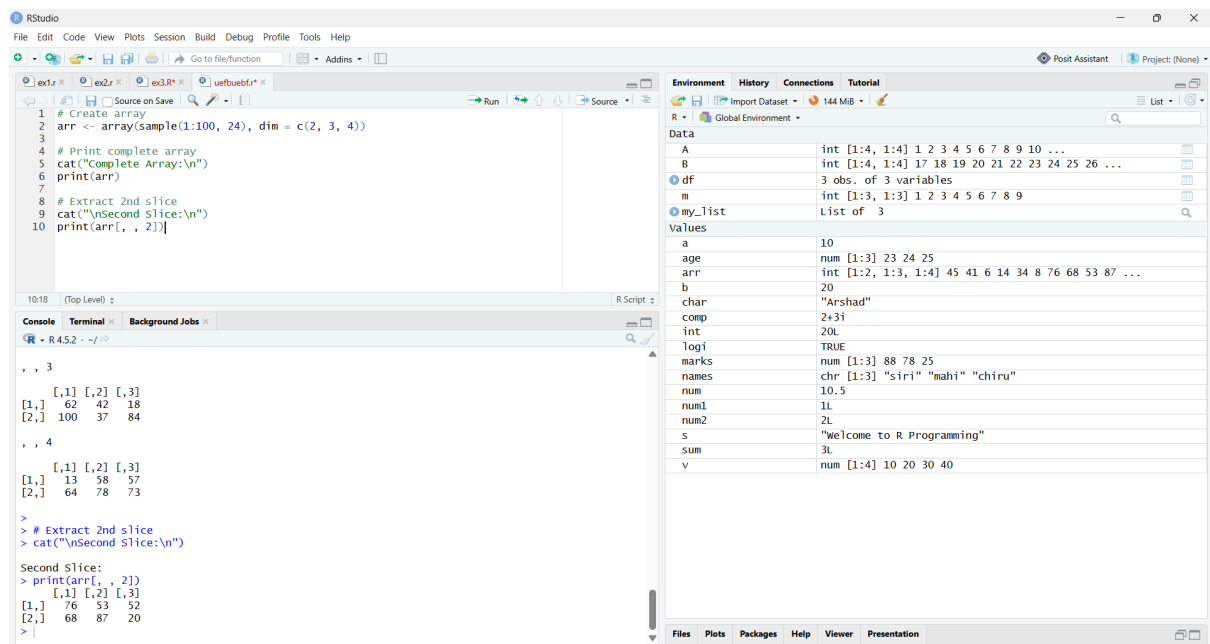
Environment

Object	Class	Attributes
A	matrix	int [1:4, 1:4] 1 2 3 4 5 6 7 8 9 10 ...
B	matrix	int [1:4, 1:4] 17 18 19 20 21 22 23 24 25 26 ...
df	data.frame	3 obs. of 3 variables
m	integer	[1:3, 1:3] 1 2 3 4 5 6 7 8 9
my_list	list	List of 3
a	numeric	10
age	numeric	[1:3] 23 24 25
b	numeric	20
char	character	"Arshad"
comp	numeric	2+3i
int	integer	20L
logi	logical	TRUE
marks	numeric	[1:3] 88 78 25
names	character	[1:3] "siri" "mahi" "chiru"
num	numeric	10.5
num1	integer	1L
num2	integer	2L
s	character	"Welcome to R Programming"
sum	integer	3L
v	numeric	[1:4] 10 20 30 40

4. 3D Array

```
arr <- array(sample(1:100,24), dim=c(2,3,4))
cat("Complete Array:\n")
print(arr)

cat("\nSecond Slice:\n")
print(arr[, , 2])
```



5. Student Data Frame

```
student <- data.frame(
  StudentID = 1:10,
  Name = c("Arun", "Bala", "Charan", "Divya", "Esha",
           "Farhan", "Gopi", "Hari", "Isha", "John"),
  Marks = c(85, 42, 67, 91, 35, 78, 55, 48, 88, 60),
  Grade = c("A", "D", "B", "A", "F", "B", "C", "D", "A", "C")
)

student$Result <- ifelse(student$Marks >= 50, "Pass", "Fail")
print(student)
```

RStudio interface showing R code execution and environment details.

Source Editor:

```
1 # Create data frame
2 student <- data.frame(
3   StudentID = 1:10,
4   Name = c("Arun", "Bala", "Charan", "Divya", "Esha",
5           "Farhan", "Gopi", "Hari", "Isha", "John"),
6   Marks = c(85, 42, 67, 91, 35, 78, 55, 48, 88, 60),
7   Grade = c("A", "D", "B", "A", "F", "B", "C", "D", "A", "C")
8 )
9
10 # Add Result column based on Marks
11 student$Result <- ifelse(student$Marks >= 50, "Pass", "Fail")
12
13 # Display the updated data frame
14 print(student)
```

Console:

```
+ Name = c("Arun", "Bala", "Charan", "Divya", "Esha",
+         "Farhan", "Gopi", "Hari", "Isha", "John"),
+ Marks = c(85, 42, 67, 91, 35, 78, 55, 48, 88, 60),
+ Grade = c("A", "D", "B", "A", "F", "B", "C", "D", "A", "C")
+ )
>
> # Add Result column based on Marks
> student$Result <- ifelse(student$Marks >= 50, "Pass", "Fail")
>
> # Display the updated data frame
> print(student)
  StudentID Name Marks Grade Result
1         1  Arun   85    A   Pass
2         2  Bala   42    D   Fail
3         3 Charan   67    B   Pass
4         4 Divya   91    A   Pass
5         5  Esha   35    F   Fail
6         6 Farhan   78    B   Pass
7         7  Gopi   55    C   Pass
8         8  Hari   48    D   Fail
9         9  Isha   88    A   Pass
10        10  John   60    C   Pass
```

Environment:

Variable	Value
a	0
age	num [1:3] 23 24 25
arr	int [1:2, 1:3, 1:4] 45 41 6 14 34 8 76 68 53 87 ...
b	1
char	"Arshad"
comp	2+3i
int	20L
logi	TRUE
marks	num [1:3] 88 78 25
n	NA_integer_
names	chr [1:3] "Siri" "mahi" "chiru"
num	10.5
num1	1L
num2	2L
s	"Welcome to R Programming"
sum	3L
v	num [1:4] 10 20 30 40

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Excellent)	Level 3 (Good)	Level 2 (Satisfactory)	Level 1 (Needs Improvement)
Data Preparation	15%	Dataset is correctly created/imported with no errors.	Dataset is mostly correct with minor errors.	Dataset contains some inaccuracies affecting calculations.	Dataset is incorrect or incomplete.
Calculation Accuracy	35%	All statistical measures/calculations are correct and accurately interpreted.	Most calculations are correct with minor errors.	Several calculation errors present.	Major errors; calculations are incorrect or missing.
Use of R Functions	20%	Appropriate R functions are selected and used efficiently.	Functions are mostly appropriate with minor issues.	Limited or partially correct use of functions.	Incorrect or no use of relevant R functions.
Analysis and Interpretation	20%	Results are clearly interpreted with logical conclusions.	Interpretation is mostly correct with minor gaps.	Limited interpretation of results.	No meaningful interpretation provided.
Presentation of Results	10%	Output is clearly organized, labeled, and documented.	Output is organized with minor formatting issues.	Output lacks clarity or proper labeling.	Output is poorly presented or incomplete.